spring
Spring的组成
（1）spring core：提供了框架的基本组成部分，包括控制反转（Inversion of Control，IOC）和依赖注入（Dependency Injection，DI）功能。
（2）spring beans：提供了BeanFactory，是工厂模式的一个经典实现，Spring将管理对象称为Bean。
（3）spring context：构建于 core 封装包基础上的 context 封装包，提供了一种框架式的对象访问方法。
（4）spring jdbc：提供了一个JDBC的抽象层，消除了烦琐的JDBC编码和数据库厂商特有的错误代码解析，用于简化JDBC。
（5）spring aop：提供了面向切面的编程实现，让你可以自定义拦截器、切点等。
（6）spring Web：提供了针对 Web 开发的集成特性，例如文件上传，利用 servlet listeners 进行 ioc 容器初始化和针对 Web 的 ApplicationContext
（7）spring test：主要为测试提供支持的，支持使用JUnit或TestNG对Spring组件进行单元测试和集成测试。
spring事务
特性 ACID
接口
PlatformTransactionManager 
TransactionStatus getTransaction(TransactionDefinition definition) 
void commit(TransactionStatus status)
void rollback(TransactionStatus status)
TransactionDefinition事务属性 
事务隔离级别
事务并发
脏读（Dirty read） 
丢失修改（Lost to modify） 
不可重复读（Unrepeatableread） 
幻读（Phantom read） 
隔离级别的常量
ISOLATION_DEFAULT 
ISOLATION_READ_UNCOMMITTED 
ISOLATION_READ_COMMITTED 
ISOLATION_REPEATABLE_READ 
ISOLATION_SERIALIZABLE 
传播行为
支持当前事务
PROPAGATION_REQUIRED 
PROPAGATION_SUPPORTS 
PROPAGATION_MANDATORY 
不支持当前事务
PROPAGATION_REQUIRES_NEW 
PROPAGATION_NOT_SUPPORTED 
PROPAGATION_NEVER 
其他 PROPAGATION_NESTED 
超时
只读
回滚规则
TransactionStatus
TransactionSynchronizationManager 
QA Spring 事务实现方式有哪些？
编程式事务管理通过编程的方式管理事务，这种方式带来了很大的灵活性，但很难维护
声明式事务管理将事务管理和业务代码分离。你只需要通过注解或者XML配置管理事务
springboot
springboot启动过程 
springboot-starter机制
自动配置
bean @Configuration&与@Bean->基于java代码的bean配置
Properties @EnableConfigurationProperties与@ConfigurationProperties->读取配置文件转换为bean
Import @EnableAutoConfiguration、@AutoConfigurationPackage 与@Import->实现bean发现与加载
SPI 
SpringBootApplication接口用到的注解
@Target(ElementType.TYPE) // 注解的适用范围，其中TYPE用于描述类、接口（包括包注解类型）或enum声明
@Retention(RetentionPolicy.RUNTIME) // 注解的生命周期，保留到class文件中（三个生命周期）
@Documented // 表明这个注解应该被javadoc记录
@Inherited // 子类可以继承该注解
@SpringBootConfiguration // 继承了Configuration，表示当前是注解类
@EnableAutoConfiguration // 开启springboot的注解功能，springboot的四大神器之一，其借助@import的帮助
@ComponentScan(excludeFilters = { // 扫描路径设置
@Filter(type = FilterType.CUSTOM, classes = TypeExcludeFilter.class),
@Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication {
springboot的四大神器
starters
auto-configuration 存在于starter中，所以每个starter都是依赖autoconfigure的
cli Spring Boot CLI是一个命令行使用Spring Boot的客户端工具
运行groovy脚本 => 官网2.1
打包groovy文件到jar => 官网2.3
初始化Spring Boot项目 => 官网2.4
actuator actuator是spring boot提供的对应用系统的自省和监控的集成功能，可以对应用系统进行配置查看、相关功能统计等
QA
SpringBoot自动装配中为什么用
DeferredImportSelector
在SpringBoot自动装配中核心是会加载所有依赖中的META-INF/spring.factories文件中的配置信息。
有多个需要加载的spring.factories文件，就需要多次操作。考虑把所有的信息都加载后再统一把这些需要注入到容器中的内容注入进去
DeferredImportSelector：延迟注入Bean实例的作用
IOC
bean 的作用域
singleton : 唯一 bean 实例，Spring 中的 bean 默认都是单例的。
prototype : 每次请求都会创建一个新的 bean 实例。
request : 每一次 HTTP 请求都会产生一个新的 bean，该 bean 仅在当前 HTTP request 内有效。
session : 每一次 HTTP 请求都会产生一个新的 bean，该 bean 仅在当前 HTTP session 内有效。
global-session：全局 session 作用域，仅仅在基于 Portlet 的 web 应用中才有意义，Spring5 已经没有了
spring启动过程 spring都有哪些context
依赖注入
是什么依赖注入（Dependency Injection,DI）是实现控制反转（IoC）的一种方法，主要目的是为了降低程序中各个类之间的
耦合度。通过将依赖关系的管理交给外部容器（如Spring框架）而不是由类自己来控制，从而实现了控制反
方式
构造器注入（Constructor Injection）
属性注入（Setter Injection）
接口注入（Interface Injection）组件实现一个特定的接口，在接口中定义注入方法
注解注入（Annotation-based Injection） @Autowired、@Resource、@Inject
控制反转
是什么控制反转（IoC）是一种设计原则，可以通过多种方式实现，依赖注入（DI）是实现IoC的最常用方法之一
方式
依赖查找（Dependency Lookup）
容器负责在运行时查找并提供所需的依赖。
依赖查找可以是程序显式请求，或通过特定的机制自动完成
依赖注入（Dependency Injection）容器在对象创建时自动注入所需的依赖
上下文持有（Contextualized Dependency Lookup）
对象通过在特定上下文环境中查找依赖来获取所需的资源或对象
这种方式要求对象知道如何通过上下文环境（通常是通过实现特定的接口）来查找依赖，有时被视为一种特殊的依赖查找方式
bean生命周期
1、在IOC容器中，一个spring bean是通过类的构造器(class constractor)生成的
2、使用setter方法执行依赖注入
3、一旦依赖注入完成， `BeanNameAware.setBeanName()`被调用。在创建它的
bean factory中，该bean的名字被设置
4、调用`BeanClassLoaderAware.setBeanClassLoader(ClassLoader
classLoader)`，`BeanClassLoaderAware`用于获取Bean的类装载器(即当前bean
factory加载bean类使用的class loader)的回调类
5、`BeanFactoryAware.setBeanFactory()` BeanFactoryAware实现此接口的bean
可以获取到它们自己的Beanfactory
6、IOC容器调用`BeanPostProcessor.postProcessBeforeInitialization`
7、调用`@PostConstruct`
8、`InitializingBean.afterPropertiesSet()`
9、在xml文件中定义的 `init-method`
10、BeanPostProcessor.postProcessAfterInitialization()
11、bean实例已经准备好使用了。使用bean执行任务。
12、当`ApplicationContext`关闭时，比如使用`registerShutdownHook()`，然后
调用带有``@PreDestroy`销毁注释的方法。
13、调用`DisposableBean.destroy()`
14、调用在xml中定义的 `destroy-method` 方法
15、在垃圾回收之前，`finalize()`方法被调用
步骤
1. Bean 定义的注册（BeanDefinition）
每个 Bean 都是按照在 Spring 配置文件中定义的 BeanDefinition 来创建的。这个定义包含了类名、作用
域、生命周期回调等信息
注册：BeanDefinition 对象通常在应用启动阶段通过读取配置文件（如 XML 文件、Java 配置类或者通过注
解扫描得到的配置）注册到 BeanFactory
2. 构造器实例化根据 BeanDefinition 中的信息，Spring 使用 Bean 的构造器创建 Bean 实例。这是通过反射机制完成的
3. 属性填充
Spring 容器将在 Bean 实例化后填充所有在 BeanDefinition 中指定的属性。这包括对其他 Bean 的引用，以
及对基本类型和字符串的赋值
4. BeanNameAware 和 BeanFactoryAware 如果 Bean 实现了 BeanNameAware 或 BeanFactoryAware 接口，Spring 将调用
setBeanName(String) 或 setBeanFactory(BeanFactory) 方法，传入 Bean 的 ID 和容器的引用
5. BeanPostProcessors 的前置处理
在 Bean 初始化之前，Spring 允许 BeanPostProcessors 对象对 Bean 实例进行一些前置处理。
postProcessBeforeInitialization(Object bean, String beanName) 方法将被调用
6. 初始化方法
@PostConstruct
InitializingBean
init-method 属性
7. BeanPostProcessors 的后置处理
在 Bean 初始化之后，BeanPostProcessors 提供了另一个处理 Bean 实例的机会。
postProcessAfterInitialization(Object bean, String beanName) 方法此时被调用
8. 就绪 Bean 已完全初始化，可以被应用使用
9. 销毁方法
如果 Bean 实现了 DisposableBean 接口或在 Bean 定义中指定了 destroy-method，当应用上下文被关闭
时，这些销毁方法将被调用
关键类
BeanFactory
是什么是Spring框架中的一个核心接口，代表了Spring IoC容器的最基础形式，负责配置、创建和管理Bean，即应用中的对
象。它通过依赖注入（DI）模式来实现对象的管理和装配，是Spring实现控制反转（IoC）的基础设施
功能
Bean的创建与管理 BeanFactory负责创建和管理应用中定义的各种Bean，包括实例化、配置属性以及可能的初始化
依赖注入支持通过构造器注入和setter方法注入的方式，将依赖关系注入到组件中
按需加载 BeanFactory在默认设置下采用懒加载的策略，只有在请求获取Bean时，才会创建和初始化该Bean
实现推荐使用ApplicationContext接口的实现，如ClassPathXmlApplicationContext和AnnotationConfigApplicationContext，因为它们提供了更丰富的功能
使用场景直接使用BeanFactory的场景相对较少。大多数开发者会选择使用ApplicationContext，因为它提供了BeanFactory的所有功能外加更多扩展的企业级支持
核心方法
getBean(String name)：根据Bean的名称获取一个Bean实例。
getBean(Class<T> requiredType)：根据Bean的类型获取实例。
containsBean(String name)：检查容器中是否包含指定名称的Bean。
isSingleton(String name)：判断指定名称的Bean是否是单例。
isPrototype(String name)：判断指定名称的Bean是否是原型（多例）
getType(String name)：获取指定名称Bean的类型。
ApplicationContext
是什么
ApplicationContext
ApplicationContext extends
EnvironmentCapable, ListableBeanFactory,
HierarchicalBeanFactory,MessageSource,
ApplicationEventPublisher,
ResourcePatternResolver
ListableBeanFactory extends BeanFactory
概要
EnvironmentCapable 提供了对Environment的访问
ListableBeanFactory
目的
ListableBeanFactory是BeanFactory的扩展，添加了枚举所有bean实
例的能力，而不是客户端通过名称一个个查询bean实例
使用场景
当需要访问Spring容器中的所有bean定义时使
用，例如，对所有符合特定类型的bean执行操作
HierarchicalBeanFactory
目的
是一个扩展了BeanFactory的接口，它支持bean工厂的层次结构。这意味着一
个BeanFactory可以有父级BeanFactory，从而实现bean定义的继承
使用场景
主要用于模块化Spring应用，允许每个模块有自己的
BeanFactory，同时共享一个公共的父BeanFactory
MessageSource
目的用于国际化消息的解析，支持消息的参数化和国际化
使用场景
在需要对应用程序中的消息进行国际化处理时使
用，如根据用户的区域设置显示不同的消息
ApplicationEventPublisher
目的定义了发布事件到应用程序中所有注册监听器的方法
使用场景用于应用程序中的事件发布机制，允许组件间的松耦合通信
ResourcePatternResolver
目的
一个扩展了ResourceLoader的接口，提供了一种通过模式字符串
（如"classpath*:**/*.xml"）来解析资源（如类路径或文件系统资源）的方式
使用场景
在需要解析路径模式以定位多个资源时使用，例
如，加载配置文件或类路径下的所有资源
是Spring框架中用于提供配置信息的高级接口，继承自BeanFactory接口，因此包含BeanFactory的所有功能。它代表了Spring的IoC容器，并通过读取配置元数据
来管理和配置应用中的对象。相对于BeanFactory，ApplicationContext添加了更多的企业级功能，如事件发布、国际化支持、应用层面的上
特点
即时实例化与BeanFactory的懒加载方式不同，ApplicationContext在启动时会创建和配置所有的singleton Bean
自动Bean后处理支持自动检测并注册BeanPostProcessor和BeanFactoryPostProcessor，允许对Bean的定义和实例进行修改
自动注入支持注解方式的依赖注入，例如@Autowired、@Resource和@Inject等
事件发布机制提供了一个基于观察者模式的事件发布和监听机制
国际化支持能够通过MessageSource接口实现国际化的消息访问
环境抽象提供了对不同运行环境的抽象，可以通过Environment接口访问应用的配置信息和profiles
资源访问通过ResourceLoader接口，提供了一种统一的方式来访问多种资源，如文件、类路径资源和URL资源
Web应用支持对于Web应用，Spring提供了WebApplicationContext接口，扩展了ApplicationContext，加入了对Web应用的特殊支持
AOP集成提供了对面向切面编程（AOP）的全面支持，允许定义拦截器和切面来增强Bean的行为
实现类
ClassPathXmlApplicationContext 从类路径下的XML配置文件中加载上下文定义
FileSystemXmlApplicationContext 从文件系统中的XML文件加载上下文定义
AnnotationConfigApplicationContext
允许从一个或多个基于Java的配置类中加载Spring应用上下文
适用于基于Java配置的应用，不需要使用XML文件
XmlWebApplicationContext
专为Web应用准备，从XML文件加载配置信息
通常与Spring MVC一起使用，在web.xml中配置为ContextLoaderListener使用
AnnotationConfigWebApplicationContext
支持Web应用的Java配置类
也是与Spring MVC一起使用，在web.xml或通过代码注册Servlet时配置
对于Web应用，XmlWebApplicationContext或
AnnotationConfigWebApplicationContext通常由
Web容器（如Tomcat）在应用启动时通过
ContextLoaderListener或Servlet 3.0+的
WebApplicationInitializer接口自动创建和初始化
AbstractApplicationContext
是什么是Spring框架中ApplicationContext接口的一个抽象实现，提供了Spring IoC容器的基本功能实现。这个抽象类作为多个具体ApplicationContext实
现的基类，封装了一些共通的行为和特性，让各种具体的上下文实现能够专注于处理特定类型的配置数据（如XML配置、注解配置或其他格式）
核心功能
BeanFactory的封装它内部封装了DefaultListableBeanFactory，负责管理Bean的定义、创建和装配过程
资源访问提供了统一的资源访问方式，如对文件系统、类路径或外部URL资源的访问
事件发布机制支持应用事件的发布和监听，包括内置的事件（如上下文刷新、关闭等）和自定义事件
国际化支持通过MessageSource接口，支持应用消息的国际化。BeanFactory不支持国际化，即i18n
环境抽象提供了对运行时环境的抽象，包括配置文件和profiles
生命周期管理管理Spring IoC容器的生命周期，包括初始化、刷新和销毁过程
主要方法定义了一些重要的方法，用于
管理容器的生命周期和配置
refresh()：刷新容器，加载或刷新配置信息，实例化所有单例Bean
准备刷新：此步骤包括设置启动时间、激活状态标志以及进行一些基础的属性设置。同时，
也会完成必要的初始化前准备工作，比如初始化属性源（property sources）或者环境配
置。
获取BeanFactory：加载或获取BeanFactory。对于AbstractApplicationContext的子类来
说，这个步骤通常涉及到加载Bean定义资源（如XML文件、注解等），并将这些定义信息
载入到BeanFactory中。
BeanFactory的准备工作：对BeanFactory进行配置，准备在上下文中使用。这包括设置类
加载器、设置SpEL表达式解析器、注册一些特殊的Bean等。
Bean定义的后处理：在标准初始化之后，允许BeanFactoryPostProcessor对配置的Bean定
义进行修改。这一步是配置阶段的最后机会，可以用来覆盖或添加属性等。
初始化BeanFactory：初始化BeanFactory以进行Bean定义的注册。这包括注册各种
BeanPostProcessors。
初始化单例Bean：预实例化所有单例Bean。这确保了非懒加载的单例Bean在启动时即被创
建。这一步骤是Bean生命周期管理的核心，包括依赖注入、Bean的初始化方法执行等。
完成BeanFactory的初始化：完成BeanFactory的初始化工作，包括注册事件监听器和监听
器工厂等。
发布上下文刷新事件：发布ContextRefreshedEvent，通知相应的监听器上下文已经刷新完
成。这允许应用在容器完全初始化后执行一些特定的逻辑。
启动和注册动态更新机制：如果配置了动态更新和热部署的功能，这一步会进行必要的启动
和注册工作。
注册虚拟机关闭钩子：如果配置允许，注册一个在虚拟机关闭时自动调用close()方法的钩
子，以确保应用上下文的优雅关闭。
close()：关闭容器，释放资源并触发销毁生命周期回调
getBeanFactory()：获取内部的BeanFactory实例
publishEvent(ApplicationEvent event)：发布事件到注册的监听器
使用场景
自身是一个抽象类，因此它不直接用于实例化对
象。相反，开发者会使用它的具体子类来实现特
定类型的应用上下文。例如
ClassPathXmlApplicationContext：用于从类路径下的XML文件加载Bean定义
FileSystemXmlApplicationContext：用于从文件系统中的XML文件加载Bean定义
AnnotationConfigApplicationContext：用于基于Java注解的配置
BeanPostProcessor
是什么是 Spring 框架中的一个核心接口，它提供了两个回调方法，允许开发者在 Spring 容器的 Bean 实例化过程中，即初始化前后插入自定义逻辑。通过实
现 BeanPostProcessor 接口，可以对 Spring 管理的所有 Bean 或特定 Bean 进行额外的处理，这使得它成为自定义 Bean 行为的强大工具
主要方法
postProcessBeforeInitialization 在任何 bean 初始化回调（如 @PostConstruct 或 afterPropertiesSet）之前调用，可以返回一个对 bean 进行包装后的对象，
Bean 还未被初始化（如执行初始化逻辑或开始使用）之前，对 Bean 实例进行额外的处理
postProcessAfterInitialization
在 Bean 的初始化之后调用，可以理解为所有初始化回调之后，比如 InitializingBean 的 afterPropertiesSet 或自定义的 init 方法执行完毕之后
这个阶段适合执行那些需要在 Bean 完全准备好后才能执行的操作，例如用于创建代理对象的 AOP 功能
在 Spring Bean 生命周期中的角色
自定义逻辑注入允许在 Bean 初始化的前后插入自定义逻辑，这可以用于设置额外的属性，检查 Bean 的状态，或者对 Bean 进行代理等
框架集成 Spring 内部使用 BeanPostProcessor 实现了诸如 AOP 代理、事件监听器注册、资源注入等高级功能。通过这种方式，Spring 扩展了它的功
能，同时为开发者提供了一种强大的扩展机制
条件逻辑处理可以根据 Bean 的状态或属性，在初始化前后应用条件逻辑，比如仅对特定类型的 Bean 或满足特定条件的 Bean 应用某些处理
在 Spring Boot 启动过程中的角色
自动配置 Spring Boot 利用 BeanPostProcessor 来条件性地应用自动配置，比如根据 classpath 上的类、Bean 的存在与否来决定是否配置某个 Bean
AOP 和代理 BeanPostProcessor 用于创建 AOP 代理，这是实现声明式事务管理、安全性等特性的基础
属性注入例如，@Value 注解的处理就依赖于 BeanPostProcessor，在 Bean 初始化的适当阶段注入配置属性
InstantiationAwareBeanPostProcessor
是什么 InstantiationAwareBeanPostProcessor 是 Spring 框架中的一个高级接口，继承自 BeanPostProcessor
作用在 Bean 实例化阶段之前以及之后执行自定义逻辑，提供了更细粒度的控制。这个接口在 Spring Bean 的生命周期中扮演了非常重要
的角色，特别是在实例化和属性设置阶段
主要方法
postProcessBeforeInstantiation 在目标 Bean 被实例化之前调用。返回 Bean 的代理实例，来代替真正的实例
postProcessAfterInstantiation
在 Bean 实例化之后，但在 Spring 属性填充（依赖注入）发生之前调用。这个方法可以用来干预 Bean 的属性填充过程。如果这个
方法返回 false，那么属性填充将被跳过
这个方法提供了一个机会来修改 Bean 的状态，或者提前终止属性的自动注入
postProcessProperties 在 Spring 5.1 中引入，替代了 postProcessPropertyValues，用于在 Bean 属性赋值之后执行操作。可以对 Bean 的属性进行修改或验证
在 Bean 生命周期中的角色
InstantiationAwareBeanPostProcessor 在 Spring Bean 的生命周期
中，充当了一个在实例化及初始化前后提供额外处理能力的角色。它
的存在使得开发者能够插入自定义逻辑，比如创建代理对象、阻止默
认的实例化过程、在属性注入前后进行自定义操作等
实例化前：通过 postProcessBeforeInstantiation 方法，可以用于创建代理对
象，避免了标准实例化流程。
实例化后、属性填充前：postProcessAfterInstantiation 方法可以用于决定是否
继续进行属性注入，或对实例化后的对象进行特定处理。
属性填充后：通过 postProcessProperties（或其前身
postProcessPropertyValues）方法，可以对注入的属性进行进一步处理或验证
CommonAnnotationBeanPostProcessor
是什么是作为一个 BeanPostProcessor 被注册并应用的
调用时机 refresh registerBeanPostProcessors(beanFactory);
registerBeanPostProcessors(beanFactory) 方法负责注册所有的 BeanPostProcessor，包括
CommonAnnotationBeanPostProcessor。CommonAnnotationBeanPostProcessor 的注册是在 refresh() 方法执行
的 registerBeanPostProcessors(beanFactory) 这一步骤中完成的。这个方法会遍历应用上下文中所有的
BeanPostProcessor 定义，并将它们注册到 BeanFactory 中，以便在创建和初始化 Bean 时使用
作用 CommonAnnotationBeanPostProcessor 负责处理 @PostConstruct、@PreDestroy 以及 @Resource 注解
自动注册的方式
通过组件扫描：如果你的 Spring 应用使用了组件扫描（@ComponentScan），并且 CommonAnnotationBeanPostProcessor 被标注为一个组件
（@Component），那么它会被自动注册。
通过 @EnableWebMvc：在 Spring MVC 应用中，如果你使用了 @EnableWebMvc 注解，它会间接注册 CommonAnnotationBeanPostProcessor，因为
WebMvcConfigurationSupport 配置中包含了对这个处理器的注册
在 AnnotationConfigApplicationContext 中：对于基于 Java 配置的 Spring 应用，AnnotationConfigApplicationContext 及其子类（如
AnnotationConfigServletWebServerApplicationContext）在初始化时会注册 CommonAnnotationBeanPostProcessor
默认注册：对于大多数标准的 Spring 应用上下文，CommonAnnotationBeanPostProcessor 是作为默认的 BeanPostProcessor 之一自动注册的，以支持
@PostConstruct 和 @PreDestroy 注解。
创建Bean的几种方式（x开工定配供导扫）
1. xml配置（x）
2. @ComponentScan(扫) @Componse、@Service、@Controller、@Repository
3. @Configuration（配） @Bean
4. @Import（导）
5. FactoryBean实现Bean注入（工）
6. 通过Supplier方式创建Bean（供） 1
7. starter的方式（开）在resources下创建META-INF/spring.factories文件
8. 实现特定接口，注册bean（定）
1. 实现ImportSelector或者ImportBeanDefinitionRegistrar接口
2. 1 implements BeanFactoryAware 接口，手动注入Bean实例
3. 1
实现BeanDefinitionRegistryPostProcessor接口,该接口是spring提
供的一个扩展点，用于允许开发者在应用程序上下文启动时动态
修改或注册bean定义
概要
BeanDefinitionRegistry
是一个接口，它定义了注册Bean定义（BeanDefinition）和通过Bean名字进行查询的各种方法。这个接口的主要职责
是作为Bean定义的注册中心，允许在运行时将新的Bean定义加入到注册表中或是从注册表中移除Bean定义。简而言
之，它是一个用于管理Bean定义的注册中心
DefaultListableBeanFactory
DefaultListableBeanFactory通过实现BeanDefinitionRegistry接口
它不仅实现了BeanDefinitionRegistry接口，负责Bean定义的注册，还继承自AbstractAutowireCapableBeanFactory
并间接实现了BeanFactory接口，负责根据Bean定义创建和管理Bean的实例。这意味着DefaultListableBeanFactory
既是一个Bean定义的注册中心，又是一个能够创建和管理Bean实例的工厂
ImportSelector
主要用于根据条件导入配置类。这是一种相对简单且直接的方式，适用于需要根据特定条件动态添加配置类的场景
虽然ImportSelector自身不直接操作BeanDefinitionRegistry，但通过返回的配置类名数组，Spring容器将解析这些配
置类，并将它们定义的Bean注册到BeanDefinitionRegistry中
ImportBeanDefinitionRegistrar
提供了更大的灵活性，允许开发者不仅仅是导入配置类，而且可以基于复杂逻辑动态地注册Bean定义
实现ImportBeanDefinitionRegistrar接口的类可以直接操作BeanDefinitionRegistry
QA
Bean有哪几种配置方式
XML配置
注解配置
JavaConfig方式使用Java类来配置Bean，通过编写一个配置类，使用@Configuration注解标识，然后在方法上使用@Bean注解来定义Bean
@Import @Import注解可以用于导入其他配置类，也可以用于导入其他普通类。当导入的是配置类时，被导入的配置类中定义
的Bean会被纳入到当前配置类的上下文中；当导入的是普通类时，被导入的类本身会被当作一个Bean进行注册。
Groovy配置使用Groovy脚本来配置Bean，通过编写一个Groovy脚本文件，使用Spring的DSL（Domain Specific Language）来定义Bean
JSR-330 Spring提供了对JSR-330标准注释(依赖注入)的支持，可以使用@Named 或 @ManagedBean替代@Component，但是需要javax.inject依赖
当使用@Component Bean的Name默认是什么？ Bean的名字不全是类名首字母小写，当第1个和第2个字符都是大写的时候，就不转写字符串了
Spring是如何解决Bean的循环依赖？
采用三级缓存解决的就是三个Map
一级缓存（Singleton Objects）用于存放完全初始化完成的Bean，即已经完成了实例化、属性注入和初始化方法调用的Bean
二级缓存（Early Singleton Objects）
存放提前暴露的Bean，即还没有完全初始化完成（可能还没有进行属性注入或初始化方法调用）的Bean。当
一个Bean正在创建过程中（即已经被实例化但还没有完全初始化）并且被另一个Bean依赖时，通过二级缓存
提供对这个还未完全初始化的Bean的引用，可以使依赖它的Bean得以继续创建。这个缓存在Spring中通常被
称为"earlySingletonObjects"
三级缓存（Singleton Factories）
存放创建Bean的工厂（ObjectFactory或getSingleton()），用于生成Bean的代理对象。用于创建对象并放入二级
缓存中。同时，如果对象有Aop代理，则对象工厂返回代理对象。针对那些涉及到循环依赖的Bean，Spring容器才
会将创建这些Bean的工厂对象放入三级缓存中，那么这个工厂可以用来生成Bean的早期引用（early reference），
这个早期引用随后会被放入二级缓存中，以供其他依赖它的Bean使用
Bean创建和三级缓
存应用的简化流程
1. 实例化Bean
Spring首先会调用Bean的构造方法来创建Bean的实例。这一步骤发生在Bean生命周期的最
初阶段，此时Spring尚未考虑将Bean放入任何缓存中
这一步还未涉及到任何缓存
2. 提前暴露引用（解决循环依赖）
一旦Bean实例被创建，如果检测到存在循环依赖，Spring会将这个尚未
完全初始化的Bean实例提前暴露出来，以便其他Bean引用。这个提前
暴露的Bean实例会被放入三级缓存（singletonFactories），这里存放的
是一个能够生成Bean代理或提前引用的工厂对象
三级缓存（singletonFactories）
3. 属性填充和初始化
在Bean实例化之后，Spring会进行属性的填充（依赖注入）并调用初始化方法。这一
步骤通常发生在Bean完全进入一级缓存之前
一步主要是Bean的创建流程，尚未涉及到缓存的
转移或使用
4. 处理AOP代理
如果Bean需要被代理（比如通过AOP），并且存在循环依赖，那么通过三级缓存中的工厂
对象来生成代理对象。这样做允许在Bean完全初始化之前就可以获取到其代理对象的引用
二级缓存（earlySingletonObjects）与三级缓存
（singletonFactories）的交互
5. 成初始化并放入一级缓存
Bean完成初始化后，如果它被提前暴露过，则其代理对象或原始对象会从三级
缓存转移到一级缓存中，此时Bean已经可以被容器正常使用了
一级缓存（singletonObjects）
一旦Bean完全初始化（包括依赖注入和初始化方
法调用），它会从三级缓存（如果提前暴露过）
转移到一级缓存中。如果在此过程中创建了AOP
代理，那么代理对象而非原始Bean实例会被放入
一级缓存。此时，Bean已完全准备好被容器使用
二级缓存能不能解决循环依赖？
如果只是循环依赖导致的死循环的问题：一级缓存就可以解决，但是解决在并发下获取不完整的Bean
如果要使用二级缓存解决循环依赖，意味着所有Bean在实例化后就要完成AOP代理，这样违背了Spring设计
的原则，Spring在设计之初就是通过AnnotationAwareAspectJAutoProxyCreator这个后置处理器来在Bean
生命周期的最后一步来完成AOP代理，而不是在实例化后就立马进行AOP代理。
如果 Bean 本身不需要代理的话，是直接返回原
始 Bean 的，并不需要走复杂的创建代理 Bean 的
流程。
Spring有没有解决多例Bean的循环依赖？
 a. 多例不会使用缓存进行存储（多例Bean每次使用都需要重新创建）
 b. 不缓存早期对象就无法解决循环
Spring有没有解决构造函数参数Bean的循环依赖？
 a. 构造函数的循环依赖也是会报错
 b. 可以通过人工进行解决：@Lazy
 ⅰ. 就不会立即创建依赖的bean了
 ⅱ. 而是等到用到才通过动态代理进行创建
对循环依赖的处理有三种情况
①构造器的循环依赖：这种依赖spring是处理不了的，直接抛出BeanCurrentlylnCreationException异常。
②单例模式下的setter循环依赖：通过“三级缓存”处理循环依赖。
③非单例循环依赖：无法处理
解释Spring中bean的生命周期分4大步
实例化
 a. 通过反射去推断构造函数进行实例化
 b. 实例工厂、静态工厂
依赖注入（DI） 解析自动装配（byname bytype constractor none @Autowired）
初始化
 a. 调用很多Aware回调方法
 b. 调用BeanPostProcessor.postProcessBeforeInitialization
 c. 调用生命周期回调初始化方法
 d. 调用BeanPostProcessor.postProcessAfterInitialization, 如果bean实现aop则会在这里创建动态代理
销毁
 a. 在spring容器关闭的时候进行调用
 b. 调用生命周期回调销毁方法
BeanFactory 和FactoryBean有什么区别 FactoryBean
是一个bean，但是它是一个特殊的bean
FactoryBean接口定义了两个方法：getObject()
和getObjectType()。
getObjectType()方法用于返回创建的Bean对象的类型。
getObject()方法用于返回创建的Bean对象，最终该Bean对象会进行注入，MyBatis集成Spring时的那个
SqlSessionFactoryBean就实现了FactoryBean，最终通过getObject()将SqlSessionFactory注入到Ioc容器中。
要获得FactoryBean类本身而非getObject()返回的Bean可以通过在BeanName前加“&”进行获取
Spring-Ioc容器的加载过程
配置解析阶段加载和解析配置文件，将配置
的bean解析成 BeanDefinition
1. 读取配置：通过BeanDefinitionReader读取配置文件或配置类
2. 解析配置信息：如ComonentScan、Bean配置等
3. 扫描类注解：根据ComonentScan扫描@Component、@Bean、@Configuration、@Import等注解...
4. 将符合的bean注册为BeanDefinition
Bean 的创建段根据 BeanDefinition创建Bean
实例化Bean：容器根据配置文件中的Bean定义，实例化Bean对象。可以通过构造函数实例化、工厂方法实例化、静态工厂方法实例化等方式来创建Bean对象
注入Bean属性：容器会为实例化的Bean对象设置属性值，可以通过setter方法注入属性值，也可以通过构造函数注入属性值。
处理依赖关系：容器会处理Bean之间的依赖关系，将依赖的Bean注入到需要的地方。
执行初始化方法：容器会调用Bean的初始化方法，可以通过实现InitializingBean接口或在配置文件中指定初始化方法来定义Bean的初始化逻辑。
注册Bean：容器会将实例化、属性设置和初始化完成的Bean对象注册到容器中，以便后续的使用和管理。
完成加载：容器完成所有Bean的加载和初始化后，即完成了IoC容器的加载过程。此时，可以通过容器调用getBean获取Bean对象。
spring 自动装配 bean 有哪些方式
根据名称自动装配（byName）
根据类型自动装配（byType）
构造函数自动装配（constructor）
自动装配注解（Autowired）
Spring框架中的单例bean是线程安全的吗
如果单例Bean的状态是可变的，并且多个线程同时修改该状态，可能会导致线程安全问题
措施
避免在单例Bean中使用可变的实例变量，或者确保对这些变量的访问是线程安全的
尽量避免在单例Bean中使用共享的外部资源，如数据库连接、文件等。如果必须使用共享资源，需要确保对这些资源的访问是线程安全的（如
使用ThreadLocal等）来保证线程安全
使用无状态的单例Bean。无状态的单例Bean不包含任何实例变量，只包含方法和局部变量
采用多例Bean。将bean的作用域改为"Prototype"即每次使用创建一个新的实例
BeanFactory 和 ApplicationContext有什么区别？
BeanFactory
Bean的生产过程分为【配置的解析】和【Bean的创建】，而BeanFactory只有Bean的创建功能
基础设施 BeanFactory是Spring框架中的最基础的容器，提供了基本的DI支持
按需加载它在默认情况下是懒加载的，即只有在请求时才创建Bean实例
功能有限相比于ApplicationContext，它缺少许多企业级的支持，如事件发布、国际化消息支持、环境抽象等
ApplicationContext
比 BeanFactory 多了更多的功能
例如配置的读取、解析、扫描等，还加入了如事件事件监听机制，及后置处理器让Spring提升了扩展性
已经不是 BeanFactory 之类的工厂了，而是 “应用上下文”，代表着整个大容器的所有功能
该接口定义了一个 refresh 方法
高级特性 ApplicationContext是BeanFactory的子接口，提供了更加全面的容器功能。它支持国际化、事件传播、资源加载等高级企业级特性
立即加载 ApplicationContext在启动时会创建并配置所有的Singleton Bean。这种预加载策略确保了所有的Bean在使用之前都已经可用
AOP集成提供了对面向切面编程（AOP）的全面支持，允许定义拦截器和切面来增强Bean的行为
环境抽象提供了一个强大的环境抽象，允许访问配置文件以及环境变量等，便于应用配置的外部化和管理
适用场景由于其丰富的功能，ApplicationContext更适合企业级的应用开发，尤其是那些需要高级特性如事件通知、声明式事务管理等的场合
如何使用 BeanPostProcessor 和
BeanFactoryPostProcessor ？
Spring中出现同名bean怎么办？
同一个配置文件内同名的Bean，以最上面定义的为准
不同配置文件中存在同名Bean，后解析的配置文件会覆盖先解析的配置文件
同文件中ComponentScan和@Bean出现同名Bean。同文件下@Bean的会生效，@ComponentScan扫描进来不会
生效。通过@ComponentScan扫描进来的优先级是最低的，原因就是它扫描进来的Bean定义是最先被注册的
Spring与SpringMVC父子容器的关系
Spring容器为父容器，SpringMVC为子容器
在Spring父容器中注册的Bean对于SpringMVC容器中是可见的，而在SpringMVC容器中注册的Bean对于Spring父容器中是不可见的
这种结构允许在Web应用中分层管理bean，其中根应用上下文充当全局共享的容器，而Servlet应用上下文则提供了更细粒度的控制，适用于Web组件
Spring父容器负责所有其他非@Controller注解(如@Service，@Respository)的Bean的注册，而SpringMVC只负责@Controller注解的Bean的注册，使得
他们各负其责、明确边界
@Autowired 的实现原理？ 实现@Autowired 的关键是：AutowiredAnnotationBeanPostProcessor
在spring管理bean生命周期的时候，每个class都
是一个BeanDefinition吗？
BeanDefinition是Spring用来管理和配置Bean的方式，每个被Spring容器管理的Bean都会有一个对应的BeanDefinition对象，用于定义Bean的配置
元数据，这个对象包含了创建和管理这个Bean所需的所有信息，包含了Bean的类名、是否为懒加载、依赖关系、生命周期回调等信息
当你在Spring的配置文件中定义一个Bean，或者通过注解（如@Component, @Service, @Repository, @Controller等）标注一个类时，Spring容器
会为这个Bean创建一个BeanDefinition对象。这个对象包含了创建实例所需的所有信息，如构造函数参数、属性值、所属的类等
但并非每个class都会成为一个Bean，也并非每个class都对应一个BeanDefinition。只有当一个class被定义为Spring管理的Bean时（无论是通过
XML配置、Java配置还是注解），Spring容器才会为其创建一个BeanDefinition
AOP
AOP是什么 AOP又叫面向切面编程，旨在通过允许横切关注点的分离，提高模块化。通俗理解就是，将那些与业务无关，却为业务模块所共
同调用的逻辑代码封装起来，形成一个切面，使原来的业务功能更加强大，即增强
AOP实现方式
编译时增强（静态代理） AspectJ
实现机制
AspectJ 使用注解和编译器（织入器）在编译时将切面代码注入到目标类中。它支持编译时织入（CTW）和加载
时织入（LTW），增加了在类加载到 JVM 时织入切面的能力
Spring AOP使用了AspectJ的注解来定义切面、切入点、通知增强等，但是没有用AspectJ的代理类生成实现（所以不需要额外的AspectJ编译器或织入器
类型
编译时（编译期织入）增强
加载时（加载期织入）增强加载时织入（LTW）允许 AspectJ 在类被加载进 JVM 时动态地应用切面，这需要配置 JVM 使用特定的类
加载器。这意味着，尽管织入发生在运行时，但它发生在类加载阶段，而不是类加载之后的执行阶段
优点
强大的切面编程能力，支持复杂的切面定义和细粒度的控制
由于在编译时或加载时完成增强，运行时性能更高
其它 Spring AOP虽然使用了Aspect的Annotation（@Aspect，@Pointcut，@Before等），但是并没有使用它的编译器和织
入器。其实现原理是JDK动态代理和CGLIB，在运行时生成代理类
运行时增强
（动态代理）
JDK 动态代理
实现机制通过 Java 的反射机制和 Proxy 类，动态创建实现了目标接口的代理对象，并通过 InvocationHandler 接口拦截代理对象上的方法调用实现增强
使用条件
必须实现InvocationHandler接口；
使用Proxy.newProxyInstance产生代理对象；
被代理的对象必须要实现接口
类型运行时增强
特点
只能代理接口（需要被代理类实现某个接口）
通过反射方式生成代理类
默认使用JDK动态代理，当目标类没有实现接口时，使用CGLIB
优点
无需修改源代码，减少了对业务代码的侵入
相对于 CGLIB，性能开销较低，因为是基于接口的代理
缺点因为通过反射方式，所以比cglib性能差
CGLIB
实现机制通过底层的字节码技术(ASM,开源的Java字节码编辑库，操作字节码)实现的，它可以
在运行时动态生成目标对象的子类，并在子类中覆盖非final的方法
对指定的类生成一个子类，覆盖其中的方法，并
覆盖其中方法的增强
类型运行时增强
特点
能代理接口和类，但是不能代理被final修饰的类
通过继承被代理类的方式生成代理类（也即被代理类的子类）
优点性能好于JDK动态代理
缺点不能代理被final修饰的类
其它
Spring 5.x 中 AOP 默认依旧使用 JDK 动态代理
SpringBoot 2.x 开始，为了解决使用 JDK 动态代理可能导致的类型转化异常而默认使用 CGLIB
QA
java1.8的时候JDK的效率已高于cglib，那么它是
做了哪些优化
方法句柄（Method Handles）和调用点
（InvokeDynamic）
Java 1.7 引入了方法句柄（Method Handles）和调用点（InvokeDynamic）指令，Java 1.8 继续在此基础上进行优化。方法句柄提供了对方法、构
造函数和字段的强类型引用，而 InvokeDynamic 指令则允许在运行时动态绑定方法调用，这两者结合使用大大减少了动态代理调用的开销
Lambda表达式 Java 1.8 引入的 Lambda 表达式背后也使用了 InvokeDynamic 指令，进一步提升了 JVM 在执行动态代码时的性能。虽然 Lambda 表达式本身与动态代理的实
现直接关系不大，但是 Lambda 表达式的引入和优化反映了 JVM 在处理动态代码方面的整体性能提升，这也间接地提高了动态代理的效率
类加载和字节码生成的优化 JDK 动态代理的实现机制涉及到运行时生成代理类的字节码并加载到 JVM 中。Java 1.8 对类加载器和字节码处理进行了优化，使得动态生成的代
理类的加载和执行更加高效。这包括改进了类加载器的性能，优化了字节码验证和解析过程，从而减少了动态代理的开销
HotSpot JVM 的优化 HotSpot JVM 不断地在优化 JIT（Just-In-Time）编译器，使得 Java 代码的运行时性能得到持续提升。这些 JIT 编译器的优化，如内联优化、循环
展开、死代码消除等，都有助于提升动态代理调用的效率
其他底层优化包括但不限于改进的垃圾收集算法、线程调度机制的优化、以及更好的 CPU 利用率等，都间接地提升了 JDK 动态代理的性能
Spring-AOP通知和执行顺序？
5种类型的通知
1. 前置通知：在目标方法被调用之前调用通知功能
2. 后置通知：在目标方法完成之后调用通知，此时
不会关心方法的输出是什么
3. 返回通知：在目标方法成功执行之后调用通知
4. 异常通知：在目标方法抛出异常后调用通知
5. 环绕通知：通知包裹了被通知的方法，在被通知
的方法调用之前和调用之后执行自定义的行为
Spring AOP使用责任链模式来管理通知的执行顺序。通
知拦截链包括前置通知、后置通知、异常通知、最终通知
和环绕通知，它们按照配置的顺序形成链式结
执行顺序
Spring在5.2.7之前
正常执行前置--->方法--->后置---->返回
异常执行前置--->方法--->后置---->异常
Spring在5.2.7之后
正常执行前置--->方法---->返回--->后置
异常执行前置--->方法---->异常--->后置
什么情况下AOP会失效,怎么解决
内部方法调用
如果在同一个类中的一个方法调用另一个方法，AOP通知可能不会触发，因为AOP通常是通过代理对象拦截外部方法调用的
解决方式是注入本类对象进行调用，或者设置暴露当前代理对象到本地线程，可以通过AopContext.currentProxy() 拿到当前正在调用的动态代理对象
静态方法 AOP通常无法拦截静态方法的调用，因为静态方法不是通过对象调用的
AOP配置问题错误的AOP配置可能导致通知不正确地应用于目标方法
代理问题如果代理对象不正确地创建或配置，AOP通知可能无法生效。解决方法是调试底层源码确保代理对象正确创建，并且AOP通知能够拦截代理对象的方法调用
JDK动态代理和CGLIB动态代理的区别
性能
JDK动态代理要求目标对象必须实现至少一个接口，因为它基于接口生成代理类
CGLIB动态代理不依赖于目标对象是否实现接口，可以代理没有实现接口的类，它通过继承或者代理目标对象的父类来实现代理
从创建代理时的性能对比
JDK动态代理通常比CGLIB动态代理创建速度更快，因为它不需要生成字节码文件
CGLIB动态代理的创建速度通常比较慢，因为它需要生成字节码文件
从调用时的性能对比
JDK动态代理在方法调用时需要通过反射机制来调用目标方法，因此性能略低于CGLIB
CGLIB动态代理在方法调用时不需要通过反射，直接调用目标方法，通常具有更高的方法调用性能，同时无需类型转换
Spring AOP and AspectJ AOP 有什么区别
关系
当在Spring中要使用@Aspect、@Before.等这些注解的时候，就需要添加AspectJ相关依赖
Spring Aop主要用到了AspectJ的@PointCut 切点解析能力和切点匹配能力。 @Aspect、@Before.等这些注解都是由AspectJ 发明的
AspectJ 能干很多 Spring AOP 干不了的事情，它是 AOP 编程的完全解决方案。Spring AOP 致力于解决的是企业级开发中最普遍的 AOP 需求
（方法增强），而不是力求成为一个像 AspectJ 一样的 AOP 编程完全解决方案
区别
Spring AOP 基于动态代理方式实现；AspectJ 基于静态代理方式实现，AspectJ 主要通过在编译阶段生成代理类，也称为编译时增强；
Spring AOP 仅支持方法级别的 PointCut；AspectJ 提供了完全的 AOP 支持，它还支持属性级别的 PointCut
SpringAOP并没有使用AspectJ的织入Java字节功能来增强对象。而是使用的动态代理来增强对象。Spring AOP 在容器启动的时候需要生成代理实例，在方法调用上也会增加栈
的深度，使得 Spring AOP 的性能不如 AspectJ 那么好
aspectj和cglib区别
AspectJ 和 CGLIB 是两种不同的面向切
面编程（AOP）框架，它们在执行代理
时的时机和方法有所不同。
AspectJ
作为一套独立的面向切面编程的解决方案，AspectJ 在编译时自动编译得到一个新的类，这个新类增强了原先类的功能，因此它是编译时增强的AOP框架
使用Aspect来定义切面，使用Pointcut来定义切入点，使用Advice来定义增强处理
虽然使用了Aspect的Annotation，但是并没有使用它的编译器和织入器
CGLIB
CGLIB（Code Generation Library）是一个代码生成类库，它可以运行时动态生成某个类的子类
CGLIB包的底层使用的是ASM（Java字节码操控框架），通过转换字节码并生成新的类（运行时生成）来进行代理增强
CGLIB代理模式可以针对接口与普通类（继承方式），但在使用静态或final方法的场景下可能会遇到局限性
AspectJ是在编译时进行的增强，而CGLIB是在运行时进行的增强
springMVC SpringMVC处理请求的流程(DHHMVV)
接收请求用户发送的HTTP请求首先被前端控制器（DispatcherServlet）接收。DispatcherServlet是Spring MVC的核心控制器，负责将请求分发给相应的处理器
查找处理器映射 DispatcherServlet查询一个或多个HandlerMapping来确定请求的目标处理器（Controller）。HandlerMapping基于请求的URL、HTTP方法等信息来识别哪个控制器
应该处理请求
调用处理器适配器一旦找到目标处理器，DispatcherServlet需要一个HandlerAdapter来执行处理器（Controller）。HandlerAdapter负责调用处理器的方法，并且处理方法的返回值
处理器处理请求控制器（Controller）接收请求并调用相应的服务逻辑，然后返回一个模型（Model）和视图名称（View name）或者一个ModelAndView对象，其中包含了模型数据
和视图名称
解析视图 DispatcherServlet使用ViewResolver来解析控制器返回的视图名称到一个具体的View实例。ViewResolver根据视图名称确定如何渲染响应
渲染视图 View负责渲染模型数据，生成最终的HTML、PDF或其他格式的响应内容。在这个步骤中，模型数据被放入请求属性中，以便视图可以使用模型中的数据
返回响应渲染完成后，响应被发送回客户端
QA
SpringMVC的拦截器和过滤器有什么区别？执行顺序？
归属不同拦截器是SpringMVC框架的一部分，而过滤器是Servlet规范的一部分
执行顺序也不同拦截器的执行顺序由配置文件中的顺序决定，可以有多个拦截器，它们按照配置的顺序依次执行。而过滤器的执行顺序由web.xml文件中
的配置顺序决定，同样可以有多个过滤器，按照配置的顺序执行。一般来说，首先执行过滤器，然后再执行拦截器。
用途不同拦截器用于对SpringMVC的请求和响应进行特定的业务处理，通常与控制器层的请求处理有关。过滤器用于对所有Servlet请求和响应进行通用性
的处理，通常关注请求和响应内容，而不涉及具体的业务逻辑。
Spring和SpringMVC为什么需要父子容器
划分功能边界清晰地定义每个容器的职责
规范整体架构我们可以将业务逻辑层（Service）和数据访问层（DAO）交给Spring管理，而将控制器层（Controller）交给SpringMVC管理
限制组件之间的依赖关系子容器可以访问父容器中的组件，但反之则不成立
方便切换子容器如果我们需要更改子容器，例如从Spring MVC切换到Struts，只需更改子容器的配置而无需更改父容器
父子容器的使用还有助于节省资源父容器中的Bean可以在整个应用程序范围内共享，而不必每次创建
Spring 框架中都用到了哪些设计模式
简单工厂 BeanFactory：Spring的BeanFactory充当工厂，负责根据配置信息创建Bean实例。它是一种工厂模式的应用，根据指定的类名或ID创建Bean对象
工厂方法 FactoryBean：FactoryBean接口允许用户自定义Bean的创建逻辑，实现了工厂方法模式。开发人员可以使用FactoryBean来创建复杂的Bean实例
单例模式 Bean实例：Spring默认将Bean配置为单例，确保在容器中只有一个共享的实例，这有助于节省资源和提高性能
适配器模式
 SpringMVC中的HandlerAdapter：SpringMVC的HandlerAdapter允许不同类型的处理器适配到处理器接口，以实现统一的处理器调用。这是适配器模式的应用
Spring AOP 的增强或通知(Advice)使用到了适配器模式
装饰器模式 Spring中用到的包装器模式在类名上有两种表现：一种是类名中含有Wrapper，
另一种是类名中含有Decorator。
装饰器模式强调的是增强自身，在被装饰之后你能够在被增强的类上使用
增强后的功能。增强后你还是你，只不过能力更强了而已
代理模式 AOP底层：Spring的AOP（面向切面编程）底层通过代理模式来实现切面功
能，包括JDK动态代理和CGLIB代理。
代理模式强调要让别人帮你去做一些本身与你业
务没有太多关系的职责（记录日志、设置缓存）
观察者模式 Spring 事件驱动模型就是观察者模式很经典的一个应用
策略模式 excludeFilters、includeFilters：Spring允许使用策略模式来定义包扫描时的过滤策略，如在@ComponentScan注解中使用的excludeFilters和includeFilters
模板方法 Spring几乎所有的外接扩展：Spring框架的许多模块和外部扩展都采用模板方法模式，例如JdbcTemplate、HibernateTemplate等
责任链模式 AOP的方法调用：Spring AOP通过责任链模式实现通知（Advice）的调用，确保通知按顺序执行
委托者模式 DelegatingFilterProxy，整合Shiro，SpringSecurity的时候都有用到
命令模式 JdbcTemplate
Spring事务的失效原因
方法是private也会失效，解决：改成public
目标类没有配置为Bean也会失效，解决：配置为Bean
自己捕获了异常，解决：不要捕获处理
使用CGLIB动态代理，但@Transactional声明在接口上 pring的事务代理使用基于接口的JDK动态代理。如果您将@Transactional注
解声明在接口上，而目标类是使用CGLIB代理的，事务将不会生效。
跨越多个线程的事务管理,解决：使用编程式事务或分布式事务
事务传播属性或捕获异常等熟悉设置不正确事务传播属性定义了事务如何传播到嵌套方法或外部方法。如果事务传播属性设置不正确，可能会导致事务失效或不符合预期的行为
Spring Boot、Spring MVC
和 Spring 有什么区别？
Spring Spring 框架就像一个家族，有众多衍生产品例如 boot、security、jpa等等；但他们的基础都是Spring 的ioc和 aop，ioc 提供了依赖注入的容器， aop解决了面向切面编程，然后在此
两者的基础上实现了其他延伸产品的高级功能
Spring MVC
Spring MVC提供了一种轻度耦合的方式来开发web应用；它是Spring的一个模块，是一个web框架；通过DispatcherServlet, ModelAndView 和 View Resolver，开发web
应用变得很容易；解决的问题领域是网站应用程序或者服务开发——URL路由、Session、模板引擎、静态Web资源等等
Spring Boot
Spring Boot实现了auto-configuration自动配置（另外三大神器actuator监控，cli命令行接口，starter依赖），降低了项目搭建的复杂度。它主要是为了解决使用Spring框架
需要进行大量的配置太麻烦的问题，所以它并不是用来替代Spring的解决方案，而是和Spring框架紧密结合用于提升Spring开发者体验的工具；同时它集成了大量常用的第
三方库配置(例如Jackson, JDBC, Mongo, Redis, Mail等等)，Spring Boot应用中这些第三方库几乎可以零配置的开箱即用(out-of-the-box)
同一个类中调用 @Transaction
注解的方法会有事务效果吗？
没有，可以Autowired注入自己，然后再调用注入的类中的方法，即自己依赖自己，循环依赖；这里在一个内部调用应该是相当于
单纯的调用方法this.methodName()，并没有AOP代理
Autowired和Resource关键字的区别？
@Autowired为Spring提供的注解，需要导入org.springframework.beans.factory.annotation.Autowired;只按照byType注入
@Resource默认按照ByName自动注入，由J2EE提供，需要导入包javax.annotation.Resource
自由主题